



AWS Messaging (SQS & SNS) — Complete Notes

Decoupling · Queues, pub/sub & event fan-out

A reader-friendly, all-in-one reference for Amazon SQS & SNS.

1. Why Messaging (Decoupling)

Messaging services let components communicate **asynchronously** so they don't depend on each other being available at the same time. A producer drops a message; a consumer picks it up later. This **decouples** services → independent scaling, fault tolerance, and smoothing of traffic spikes.

💡 **Mental model: SQS = a queue** (one message → one consumer, pull-based). **SNS = a megaphone** (one message → many subscribers, push-based pub/sub). Combine them for **fan-out**.

Common uses: buffering work, decoupling microservices, fan-out notifications, smoothing spikes, event-driven architectures, retry/back-pressure handling.

2. SQS — Core Concepts

Amazon SQS (Simple Queue Service) is a fully managed **message queue**. Producers send messages; consumers **poll** and process them.

Concept	Description
Queue	Buffer holding messages until consumed.
Message	Up to 256 KB of text (larger via S3 + extended client).
Producer / Consumer	Sends / polls-and-processes messages.
Visibility Timeout	After a consumer receives a message it's hidden for this window; if not deleted in time it reappears (default 30s, max 12h).
Retention	Messages kept 1 min– 14 days (default 4 days).
Delete	Consumer must explicitly delete after processing (at-least-once).

⚠️ SQS is **pull-based** — consumers ask for messages. A message stays until a consumer **deletes** it; if processing fails it becomes visible again for retry.

3. 🔄 SQS Standard vs FIFO

	Standard	FIFO
Throughput	Nearly unlimited	300 msg/s (3,000 with batching)
Ordering	Best-effort (may arrive out of order)	Strict ordering within a group
Delivery	At-least-once (rare duplicates)	Exactly-once processing
Name	any	must end in .fifo
Use	Max throughput, order not critical	Order/no-duplicates required (payments, sequences)

- FIFO uses a **MessageGroupId** (ordering scope) and **MessageDeduplicationId** (dedupe within 5 min).

4. ✨ SQS Key Features

- **Dead-Letter Queue (DLQ)** — after N failed receives (`maxReceiveCount`), a "poison" message is moved to a separate queue for inspection — keeps the main queue flowing.
- **Long polling** (`WaitTimeSeconds` up to 20s) — waits for messages instead of returning empty → fewer empty receives, lower cost. Prefer over short polling.
- **Visibility timeout** — set it longer than your processing time; extend with `ChangeMessageVisibility` for long jobs.
- **Delay queues** — postpone delivery of new messages (0–15 min).
- **Batching** — send/receive/delete up to 10 messages per call (cheaper, faster).

5. 📢 SNS — Core Concepts

Amazon SNS (Simple Notification Service) is a fully managed **pub/sub** service. A publisher sends a message to a **topic**; SNS **pushes** it to all **subscribers**.

Concept	Description
Topic	Named channel publishers send to.
Subscription	An endpoint subscribed to a topic.
Publisher	Sends a message to a topic.
Subscriber	Receives every message (push).
Standard vs FIFO topic	FIFO topics preserve order + dedupe (pair with FIFO SQS).

SNS is **push-based** and **one-to-many** — a single publish fans out to every subscriber instantly.

6. 📡 SNS Delivery & Filtering

- **Delivery protocols:** **SQS**, **Lambda**, **HTTP/S** endpoints, **email**, **SMS**, and **mobile push** (application endpoints).
- **Message filtering** — subscribers set a **filter policy** so they only receive messages whose attributes match (avoids processing irrelevant events).
- **Retries + DLQ** — failed deliveries retry per policy; undeliverable messages can go to an SNS DLQ (an SQS queue).
- **Fan-out** to many SQS queues / Lambdas from one publish.

7. ✨ Fan-out Pattern (SNS + SQS)

The canonical decoupling pattern: publish **once** to an SNS topic, which fans out to **multiple SQS queues**, each feeding a different consumer.

```

Publisher → SNS topic —┐→ SQS queue A → Service A
                        ┆→ SQS queue B → Service B
                        └→ SQS queue C → Service C
  
```

Why it's great:

- Each consumer gets its **own queue** → independent processing, retries, and DLQs.
- New consumers subscribe **without changing the publisher**.
- SQS **buffers** spikes so slow consumers aren't overwhelmed (back-pressure).

8. ⚖️ SQS vs SNS vs EventBridge vs Kinesis

Service	Model	Use when
SQS	Queue, pull, 1 consumer per message	Decouple + buffer work; reliable async processing
SNS	Pub/sub, push, many subscribers	Fan-out notifications/events to multiple endpoints
EventBridge	Event bus, content-based routing, SaaS/partner buses, schema registry	Event-driven routing with rich rules + integrations
Kinesis	Real-time streaming , ordered, replayable, multiple consumers re-read	High-volume streaming analytics, replay, ordered shards

Rough rule: **SQS** to decouple & buffer, **SNS** to fan-out, **EventBridge** to route by content/source, **Kinesis** to stream & replay big data.

9. 🗝️ Security

- **IAM policies** + resource (queue/topic) **access policies** control who can publish/subscribe/consume.
- **Encryption** at rest with **KMS** (SSE) and **TLS** in transit.
- Access privately from a VPC via **interface endpoints (PrivateLink)**.
- SNS supports **message data protection** (redact/deny sensitive data); SQS DLQs help isolate poison messages.

10. 💰 Pricing

You pay for:

1. **SQS** — per **request** (API calls; batching reduces count). First 1M/month free.
2. **SNS** — per **publish** + per delivery (HTTP/email cheap; **SMS** notably more).
3. **Data transfer out** and KMS usage if encrypted.

🔧 **Cost tips:** use **batching** + **long polling** in SQS to cut request count · filter SNS subscriptions so you don't deliver (and pay for) irrelevant messages · be mindful **SMS** is the priciest delivery channel.

11. 📖 Common CLI Commands

```
# SQS
aws sqs create-queue --queue-name jobs
aws sqs send-message --queue-url <url> --message-body "hello"
aws sqs receive-message --queue-url <url> --wait-time-seconds 20 # long polling
aws sqs delete-message --queue-url <url> --receipt-handle <rh>

# SNS
aws sns create-topic --name alerts
aws sns subscribe --topic-arn <arn> --protocol sqs --notification-endpoint <queue-arn>
aws sns subscribe --topic-arn <arn> --protocol email --notification-endpoint me@example.com
aws sns publish --topic-arn <arn> --message "deploy finished"
```

12. ✅ Best Practices

- **Decouple** producers from consumers with a queue/topic — never call services synchronously when async will do.
- Always configure a **DLQ** with a sensible `maxReceiveCount` to isolate poison messages.
- Use **long polling** + **batching** in SQS (cheaper, fewer empty receives).
- Set **visibility timeout** > processing time; extend it for long jobs.
- Use **FIFO** only when ordering/dedup is required (it caps throughput).
- Use **SNS message filtering** so subscribers get only relevant messages.
- Make consumers **idempotent** (Standard SQS is at-least-once — duplicates can happen).
- Use the **SNS→SQS fan-out** pattern to add consumers without touching the publisher.

13. 🧠 Quick Mental Model

- **SQS = queue** (pull, one consumer per message, buffer & retry). **SNS = pub/sub** (push, many subscribers, fan-out).
- SQS: **visibility timeout** hides a message while processing; **delete** to finish; failures → **DLQ**. Use **long polling + batching**.
- **Standard** (high throughput, at-least-once, best-effort order) vs **FIFO** (ordered, exactly-once, lower throughput).
- SNS pushes to **SQS / Lambda / HTTP / email / SMS**; **filter policies** limit what each subscriber gets.
- **Fan-out = SNS → many SQS queues**, each with its own consumer + DLQ.
- Pick: **SQS** decouple/buffer · **SNS** fan-out · **EventBridge** route-by-content · **Kinesis** stream/replay.
- Make consumers **idempotent**.

14. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about SQS/SNS:

- **Q: SQS vs SNS?** — SQS is a **pull-based queue** (each message processed by one consumer). SNS is **push-based pub/sub** (each message delivered to all subscribers).

- **Q: What is visibility timeout?** — After a consumer receives a message, it's hidden for this window. If the consumer doesn't delete it in time, it reappears for another consumer (enables retry).
- **Q: Standard vs FIFO queue?** — Standard = near-unlimited throughput, at-least-once, best-effort ordering. FIFO = strict ordering + exactly-once within a group, but limited throughput (name ends in `.fifo`).
- **Q: What's a dead-letter queue?** — A separate queue that receives messages after they fail processing N times, so poison messages don't block the main queue.
- **Q: What is the fan-out pattern?** — Publish once to an SNS topic that delivers to multiple SQS queues, each with its own consumer — add consumers without changing the publisher.
- **Q: Long polling vs short polling?** — Long polling waits up to 20s for a message before returning (fewer empty responses, lower cost). Short polling returns immediately.
- **Q: How do you handle duplicates?** — Standard SQS is at-least-once, so make consumers **idempotent**; or use FIFO with a deduplication ID.
- **Q: SQS vs Kinesis?** — SQS deletes a message once consumed (work queue). Kinesis is an ordered, **replayable** stream that multiple consumers can re-read — for streaming analytics.
- **Q: SNS vs EventBridge?** — SNS is simple high-throughput fan-out. EventBridge adds content-based routing rules, SaaS/partner event buses, and a schema registry.

 Notes compiled as a quick reference for Amazon SQS & SNS.